MuonCollider BIB analysis: user manual

How to run the package, where everything lives, and what comes out. Luciano Ristori, October 2026.

Master copy: `~/code/MuonCollider/docs/user_manual.pdf` (git; source `docs/src/user_manual/user_manual.tex`). The copy `_START_HERE_user_manual.pdf` in the Analysis folder is refreshed from it automatically by `./run_all` and `./run_scan`: do not edit the copy; change the source and rebuild (see `README.txt` next to it).

The 30-second version. Everything is run from *one* folder, in a Terminal:

```
cd ~/Dropbox/Documents/MuonColliderSimulation/Analysis
```

In that folder you edit plain-text settings files (names starting with `__`), then type one command:

- `./run_all` — applies smearing and cuts to the BIB and signal files and makes the highlights plots and PDF, with the BIB rejection, signal efficiencies *and the expected number of fake tracks* for your cuts and resolutions (Sections 3 and 4).
- `./run_all --headroom` — the same, plus the headroom in the three detector resolutions: $E[\#\text{fakes}]$ as each one is degraded up to $10\times$ (Section 6).
- `./run_scan` — scans the timing, pointing-angle or hit-position resolution, re-deriving the cuts at each point (Section 5).

Every run is archived, with a copy of the settings it used, in `runs/`. The newest results are always the files whose names start with _ in the Analysis folder. Nothing you do there can damage the code. Installing on a new computer, Mac or Linux: Section 8.

1 Where everything lives

There are two places: the **code**, under git, and the **simulation folder** in Dropbox, which holds the data, the settings and all results.

1.1 The simulation folder (Dropbox) — where you work

```
~/Dropbox/Documents/MuonColliderSimulation/
|-- Data/                       input ROOT n-tuples (BIB plus/minus/ipp, combined BIB, signal)
|-- Geometry/                   detector geometry XML files (MuSIC, Vertex, IT, OT)
`-- Analysis/                   <-- THE WORKING FOLDER: cd here, edit, run
    |-- __input_files_config.txt   which files in Data/ and Geometry/ to use
    |-- __cuts_config.txt          cuts per subsystem, track definition
    |-- __smearing_config.txt      detector resolutions (position, time, angle)
    |-- __scan_config.txt          what ./run_scan scans
    |-- run_all, run_scan          the two commands (tiny launchers)
    |-- _START_HERE_user_manual.pdf    this manual (auto-refreshed copy)
    |-- _highlights_<date>_<time>.pdf   PDF of the latest ./run_all
    |-- _highlights/                    its key plots (PNG) and the same PDF
    |-- _scan_time_<date>_<time>.pdf    PDF of the latest time scan
    |-- _scan_angle_<date>_<time>.pdf   PDF of the latest angle scan
    |-- _scan_position_<date>_<time>.pdf  ... and position scan
```

```
|-- step5_headroom/            ./run_all --headroom only (Sec. 6)
|-- latest_run.txt             which run the step* folders show, + its summary
|-- step1_... step4_...        all plots/tables of the latest successful ./run_all
|-- runs/                      ARCHIVE: one folder per run or scan (see Sec. 7)
`-- claude_scratch/            scratch files from work with Claude; ignore
```

1.2 The code (git) — normally you don't touch it

```
~/code/MuonCollider/            git, github.com/LucianoRistori/MuonCollider
|-- README.md                   detailed reference: n-tuple format, every step, definitions
|-- docs/user_manual.pdf        this manual (source in docs/src/user_manual/)
|-- docs/efficiency_pt_infinity.pdf   note: how efficiencies for pT -> inf are obtained
|-- run_all_steps.sh            what ./run_all does (checks, steps 1-4, archiving)
|-- run_scan.sh                 what ./run_scan does
|-- bib_common.py               shared library: reading, smearing, cuts, efficiencies
|-- make_basic_plots.py, incidence_angle_plots.py, time_of_flight_plots.py,
|   signal_overlay_*.py, apply_cuts.py, track_efficiency.py, n1_cut_plots.py
|                               the analysis steps 1-4
|-- fake_rate.py                expected number of fake tracks (Sec. 4)
|-- resolution_scan.py          the resolution scans (Sec. 5)
|-- make_highlights_pdf.py      builds the highlights PDF of a run
|-- check_configs.py, cuts_table.py, input_files.py   read and check the settings
|-- setup_workdir.sh, requirements.txt   installation (Sec. 8)
|-- templates/                  the four settings files and the launchers, for new folders
|-- docs/papers/                the papers: jinst_fake_rate/ (paper 1), muc_tracking/
|                               (paper 2), old_draft_v1.8/ (the original draft, frozen)
|-- fake_rate_framework/        analytic fake-track framework of the paper (Sec. 13-14)
`-- resolution_scans/           the original scan scripts of the paper's v1.4
```

The launchers `run_all` and `run_scan` in the Analysis folder just find the code and run it on the folder they are in. If you move the code, run `setup_workdir.sh` again (Section 8), or use `MUONCOLLIDER_CODE=/new/path ./run_all`.

2 The four settings files

All four are commented plain text: open them in any editor and read the comments, which explain every line. A summary:

`__input_files_config.txt`	`[data]`: the BIB files `bib_plus`, `bib_minus`, `bib_ipp` (optional), the combined BIB file `bib_combined`, and the `signal` muon-gun sample; `[geometry]`: the four XML files. *Names only*: files are looked up in `Data/` and `Geometry/`. The combined file is (re)built automatically when it is missing or does not match plus + minus + ipp.
`__cuts_config.txt`	`[cuts]`: a table, one row per subsystem (VXD/IT/OT, barrel and endcap), with the three cuts *time* ($\lvert t_{\text{corrected}}\rvert$, ns), *z0* ($\lvert z_0\rvert$, mm) and *pT* (minimum p_T, GeV/c), or `off`; then three *zoom* columns that only set the range of the zoomed N-1 plots. `[track]`: a muon is "found" if at least `min_hits_found` of its hits pass (`exclude_vertex_hits`: VXD hits not counted). `[signal]`: `muon_hits_only`.
`__smearing_config.txt`	Gaussian resolutions applied to the (exact) simulated hits: `[position]` `sigma_u_mm`, `sigma_v_mm`; `[time]` `sigma_t_ns`; `[angle_u]`, `[angle_v]` `sigma_deg` (pointing angle). Each section has `enabled = true/false`. `[general]` `seed` fixes the random numbers.
`__scan_config.txt`	For `./run_scan`: `parameter` (`time`, `angle` or `position`), `containment` (% of signal hits each re-derived cut keeps) and `grid` (the values to scan); its `[headroom]` section sets the ranges of `./run_all --headroom`.

Both commands check the files before running anything (typos, unknown settings, non-numbers, missing input files) and stop with a message saying what to fix. If a settings file is missing, a fresh copy is made from `templates/` and you are asked to check it. Editing a file while a run is going has no effect on that run: the files are copied at the start.

3 Case 1 — highlights plots for some data files

1. New data? Copy the ROOT files into `Data/` and put their names in `__input_files_config.txt`. (Same data as last time: skip this.)
2. Set the cuts in `__cuts_config.txt` and the resolutions in `__smearing_config.txt`.
3. Run:

```
cd ~/Dropbox/Documents/MuonColliderSimulation/Analysis
./run_all            # step 4 only: everything the PDF needs  (~1 1/4 min)
./run_all --full     # steps 1-4: all plots of the analysis   (~6 min)
```

4. Look at `_highlights_<date>_<time>.pdf` in the Analysis folder.

What the steps are. Step 1: hit maps and densities of the BIB (plus, minus, combined). Step 2: incidence angles and time-of-flight of BIB and signal. Step 3: BIB and signal overlaid (z_0, p_T, corrected time). Step 4: the cuts — BIB rejection and signal hit efficiency per subsystem, track-finding efficiency vs. p_T, N-1 plots, expected fake tracks.

What comes out. The PDF has a title page with the settings and the main results (BIB rejection factor and signal hit efficiency for $p_T \to \infty$ per subsystem, track-finding efficiency, expected fake tracks), the cuts per subsystem, and the key plots with captions quoting that run's numbers. The same results are in `summary.txt` of the run (also at the bottom of `latest_run.txt`). The `step*` folders and `_highlights/` show the latest *successful* run; a failed run never replaces them. To compare all past runs: `cat runs/*/summary.txt`.

4 Case 2 — fake-track rate for given cuts and resolutions

This is the same command as Case 1: set `__cuts_config.txt` and `__smearing_config.txt`, then `./run_all`. The expected number of fake tracks is reported

- on the title page of the highlights PDF ("Expected fake tracks"),
- in `summary.txt` and `latest_run.txt` (last line),
- in `step4_cuts/fake_rate.csv`, with the BIB hits after the cuts in each of the six layers used.

What the number is. $E[\#\text{fakes}]$ is the expected number of fake tracks in the 6-layer IT + OT barrel "tower" of the fake-rate framework. A track counts as found if at least $k =$ `min_hits_found` of its hits pass the cuts (`__cuts_config.txt`, default 5), and fakes are counted the same way: on at least k of the 6 layers. By linearity of expectation,

$$E[\#\text{fakes}] = \sum_{S,\ |S|\geq k} P_S \prod_{i\in S} n_i, \qquad n_i = \frac{\text{BIB hits after the cuts in layer } i}{\text{area of layer } i} \times W_i^2,$$

the sum running over the sets S of at least k layers (for $k = 5$: all six layers, plus the six sets that miss one layer). W_i^2 are the plane areas of the projective tower (1 m^2 outermost plane) and P_S is the probability that one random combination of hits on the layers of S passes the exact-helix fit (one-sided 3σ χ^2 cut, $p_T > 10$ GeV/c in 5 T). P_S is computed from the geometry, as the volume of the accepted track manifold (`fake_rate_framework/geometric_K.py`, validated by direct Monte Carlo). The 5-of-6 terms dominate, by about four orders of magnitude over the 6-of-6 term, which is also given in `fake_rate.csv`. A fake that also passes on a subset is counted more than once; at these densities the overlap is negligible. Values well below 1 mean fakes are negligible.

Correction (10 October 2026). Until then P came from a Monte Carlo calibration (`fake_rate_framework/real_towe` whose depth-view fit had r and z swapped; it was about 450 times too low, and only the 6-of-6 term was counted. Runs made before that date carry the old numbers.

Position resolution. P_S is computed at 0.1 mm and scales steeply with the hit position resolution: for a set of m layers, $P_S \propto (\sigma_u/0.1\,\text{mm})^{m-2}(\sigma_v/0.1\,\text{mm})^{m-2}$ ($u = r\phi$, bending view; $v = z$; $m-2$ degrees of freedom in each view). The resolutions are taken from `__smearing_config.txt` (with position smearing off, 0.1 mm is used) and are recorded in `fake_rate.csv`. Halving both resolutions divides the 6-of-6 term by $2^8 = 256$ and the 5-of-6 terms by $2^6 = 64$.

Headroom. Scaling the BIB density of all six layers by a common factor μ scales each term by μ^m ($m =$ its number of layers); one fake is reached at the μ where the sum equals 1. This factor is quoted on the title page ("1 fake at $N\times$ the BIB"), in the last line of `summary.txt`, in the `mu_one_fake_*` columns of `fake_rate.csv`, and plotted ($E[\#\text{fakes}]$ vs. μ, with the 6-of-6 term

dashed) in `fake_rate_vs_density_multiplier.png`, a page of the highlights. It tells how much more background (or looser cuts) the tracker can take before fakes matter.

Two things to remember. (1) It uses only the IT and OT barrel layers; the calibration assumes the current geometry (a warning is printed if the layer areas change). (2) There are two track-finding efficiencies. `./run_all` reports the *whole-tracker* one (all muons; found if at least `min_hits_found` of their hits pass, VXD hits not counted). `./run_scan` reports it too, and next to it the *barrel-tower* one, defined on the same six layers as $E[\#\text{fakes}]$ (muons with no IT/OT endcap hit; found if at least `min_hits_found` of their 6 barrel hits pass). They are different quantities: compare like with like.

5 Case 3 — resolution scans

1. Edit `__scan_config.txt`:

```
parameter   = time               # or: angle, position
containment = 98                 # % of signal hits kept by each re-derived cut
grid        = log 0.010 1.0 10   # time: 10 ps - 1 ns, 10 points
#  angle:  grid = linear 0 5 11   (0 - 5 deg in 0.5 deg steps)
#  any:    grid = list 0.02 0.05 0.1
#  any:    grid = factor 1 10 10  (1x-10x today's value)
```

2. Check `__smearing_config.txt` (all the resolutions that are *not* scanned) and the p_T cut in `__cuts_config.txt`.
3. Run `./run_scan` (about 15–25 s) and open `_scan_time_<date>_<time>.pdf` (or `_scan_angle_...`, `_scan_position_...`).

What a scan does at each value. The scanned resolution is applied to the hits of *all* subsystems (for angle and position, equally to u and v; with `factor`, each scaled from its own value); every other resolution is taken from `__smearing_config.txt`; the p_T cut from `__cuts_config.txt` is kept fixed. The z_0 and time cuts of every subsystem are then *re-derived* so that each keeps `containment` % of the signal hits *in its own N-1 distribution*, exactly as in the N-1 plots of `./run_all`: z_0 on the hits passing p_T and time, time on those passing p_T and z_0. The two are found together by iteration (2–4 rounds). The z_0 and time values written in `__cuts_config.txt` are *not* used. (Before October 2026 z_0 was set first, on the hits passing p_T only; late hits of curling low-p_T muons in the outer OT barrel layer then made the OT barrel z_0 cut about four times too loose.) The scan reports $E[\#\text{fakes}]$ and the track-finding efficiency for $p_T \to \infty$, both on the barrel tower, and also the whole-tracker efficiency exactly as `./run_all` defines it (Section 4).

What comes out (in `runs/<date>_<time>_scan_<parameter>/`): a PDF with a settings page, a results table, and three plots ($E[\#\text{fakes}]$, the two efficiencies, and the derived cuts of all six subsystems vs. the resolution); `scan_results.csv` with every number. Scans made before the N-1 definition (and the paper's Sections 15–16 up to version 1.5) have OT barrel z_0 cuts about four times looser, hence a higher $E[\#\text{fakes}]$; scans made before the fake probability followed the position resolution (8 October 2026) overstate $E[\#\text{fakes}]$ whenever the position resolution was not 0.1 mm (by $2^8 = 256$ at 0.05 mm).

Tip: `grid = list <value>` gives a single point: the efficiency and $E[\#\text{fakes}]$ for one resolution with containment-derived cuts, to compare with the fixed cuts of Case 2.

6 Case 4 — headroom in the detector resolutions

`./run_all --headroom` (also with `--full`) runs as usual and then a step 5: three scans like those of Case 3 — hit position, time and pointing angle, each from today's value in `__smearing_config.txt` to 10× in 10 points, the other two kept at today's values, with the z_0 and time cuts re-derived at each point. It adds about 2–3 minutes.

- `_highlights/` and the highlights PDF get one more plot, `headroom_efakes_vs_resolution.png`: E[#fakes] vs. each resolution, in three panes (bottom axis: the resolution; top axis: × today's value), with what each reaches at 10×, or where it reaches one fake.
- `step5_headroom/` has `headroom.pdf` (summary table, the same plot, the efficiencies, then each scan's own plots), `headroom_summary.csv`, and one folder per scan with its `scan_results.csv`; the summary is also at the end of `summary.txt`.

Position resolution enters twice — through the cuts and through the fake probability ($\propto \sigma_u^{m-2}\sigma_v^{m-2}$ for m layers) — time and angle only through the cuts. The ranges can be changed in a `[headroom]` section of `__scan_config.txt` (e.g. `position = factor 1 20 20`); without it, 1–10× and 98 % containment are used. In the angle scan, a step appears where the widening z_0 cut lets the late hits of curling muons into the time-cut distribution, which then opens to several ns: this is what those cuts would really do, and is kept.

7 The archive: `runs/`

Every run gets its own folder, named by the date and time it started:

`2026-10-07_161638`	a full `./run_all --full` run
`2026-10-07_161638_pdf`	a default `./run_all` (step 4 only)
`.._scan_time`, `.._scan_angle`, `.._scan_position`	a `./run_scan`
`.._FAILED`, `.._INTERRUPTED`	did not complete — see its `run_log.txt`

When a run finishes (or fails), all the files in its folder are made **read-only**, so the archive can't be changed by accident; the folder itself can still be deleted. The copies in the `step*` folders and `_highlights/` stay editable.

Each holds the settings files exactly as used, `run_log.txt` (everything printed), `code_version.txt` (git commit of the code, input files with sizes and dates), and the results (`summary.txt`, step folders, `_highlights/`; or the scan's CSV, plots and PDF). To repeat an old run, copy its settings files back into the Analysis folder and run again. The copies are read-only too: `chmod u+w __*_config.txt` makes them editable (each run also does this for you). To edit a file inside a run on purpose: `chmod u+w runs/<run>/<file>`. To rebuild the PDF of an old run: `python3 ~/code/MuonCollider/make_highlights_pdf.py runs/<run>`. Old run folders can be deleted freely; Dropbox keeps a version history.

8 Installation on a new computer

Everything is plain bash and Python: no ROOT installation is needed (the n-tuples are read with `uproot`). The same steps work on a Mac and on Linux; only step 1 differs.

What goes where. Three things, in folders of your choice:

```
~/code/MuonCollider/              the code, cloned from GitHub (step 2)
~/MuonColliderSimulation/         any name; on your own Mac it can live in Dropbox
|-- Data/                         the ROOT n-tuples (Appendix A)
|-- Geometry/                     the detector-geometry XML files (Appendix A)
`-- Analysis/                     the working folder, made by setup_workdir.sh (step 4)
```

The working folder must sit next to `Data/` and `Geometry/`. The code can be anywhere: the launchers remember where it is.

1. Git and Python 3.9 or newer. Check with `git --version` and `python3 --version`.

- *Mac:* `xcode-select --install` gives `git` and a `python3`; for a newer Python use the installer from python.org, or Homebrew (`brew install python git`).
- *Linux:* Debian/Ubuntu `sudo apt install git python3 python3-venv python3-pip`; Fedora, Alma, RHEL `sudo dnf install git python3`. On a shared machine without administrator rights, use the Python already there (`python3 --version`) if it is 3.9 or newer.

2. The code.

```
mkdir -p ~/code && cd ~/code
git clone https://github.com/LucianoRistori/MuonCollider.git
```

3. A Python environment with the packages (`numpy`, `uproot`, `awkward`, `matplotlib`, and `scipy` for the paper's scripts). A virtual environment keeps them separate from the system's Python, which Homebrew and recent Linux distributions protect (`pip` refuses with "externally-managed-environment"):

```
python3 -m venv ~/venvs/muoncollider
source ~/venvs/muoncollider/bin/activate
python3 -m pip install --upgrade pip
python3 -m pip install -r ~/code/MuonCollider/requirements.txt
```

4. The data and the working folder. Put the ROOT files in `Data/` and the XML files in `Geometry/` (which files: Appendix A), then, *with the environment of step 3 still active*:

```
mkdir -p ~/MuonColliderSimulation/Data ~/MuonColliderSimulation/Geometry
   (copy the input files into Data/ and Geometry/)
~/code/MuonCollider/setup_workdir.sh ~/MuonColliderSimulation/Analysis
```

`setup_workdir.sh` creates the working folder, copies the four settings files from `templates/` (set to the current operating point; an existing settings file is never overwritten), writes the launchers `run_all` and `run_scan`, and checks the packages and the input files, printing what is still missing.

The launchers record the code's location and the environment's Python, so later you never need to activate the environment.

5. First run.

```
cd ~/MuonColliderSimulation/Analysis
./run_all
```

The first run also builds the combined BIB file in `Data/` (a few minutes, once). Then open `_highlights_<date>_<time>.pdf`: with the template settings the track-finding efficiency is 98.0 % and $E[\#\text{fakes}] = 3.2 \times 10^{-5}$ (at least 5 of 6 layers), as in Section 4.

Later. `cd ~/code/MuonCollider && git pull` updates the code. After moving the code or the environment, run `setup_workdir.sh` again on the same folder: it rewrites only the launchers. Memory: the largest steps use about 3 GB, so 8 GB or more is comfortable. The manual and the paper are rebuilt only if you change them (LaTeX and pandoc; see the `README.txt` next to each).

9 If something goes wrong

- **"ERROR: ... fix the settings file(s)"** — nothing was run; the message lists each problem and the file it is in.
- **Python is missing packages** — the launcher is not using the environment that has them: run `setup_workdir.sh` again with that environment active (Section 8), or install them with `python3 -m pip install -r ~/code/MuonCollider/requirements.txt`.
- **A run ends in `_FAILED`** — open its `run_log.txt`; the step that failed is at the end. Your `step*` folders and `_highlights/` still show the last good run.
- **`./run_all: Permission denied`** — the launcher lost its executable flag (e.g. after a copy): `chmod +x run_all run_scan`.
- **Help on the commands** — `./run_all --help`, `./run_scan --help`.
- **A full run under a time limit** can be split: `./run_all --full --steps 1,2`, then `./run_all --run <run> --steps 3,4`.

10 Where to read more

- `README.md` in the code folder: the n-tuple format, each analysis step and its plots, how every quantity is defined and quoted.
- `docs/efficiency_pt_infinity.pdf`: the efficiencies for $p_T \to \infty$ (the fit of $\varepsilon_\infty + c/p_T^2$).
- `docs/papers/`: the papers. `jinst_fake_rate/` is paper 1 (the method), `muc_tracking/` is paper 2 (the Muon Collider application); `old_draft_v1.8/` is the original single draft (`paper_draft.pdf`), superseded and kept frozen, with the data, settings and figures behind its Sections 14–16 (`README.txt`). The project notes (milestones, decisions) are in the Claude project "Muon Collider Tracking".

- `fake_rate_framework/`: the scripts behind the paper's Sections 13–14 (the fake-rate calibration and its plots); `resolution_scans/`: the original scan scripts of its version 1.4 (Sections 15–16 are now made with `./run_scan`).
- Changing the code: commit with git in `~/code/MuonCollider` and `git push` to GitHub; each run records the commit it used.

A The input files

Their names are set in `__input_files_config.txt`; the files are looked up in `Data/` and `Geometry/` next to the working folder.

`[data]`: ROOT n-tuples, all with one tree, `HTAtree`:

- `bib_plus`, `bib_minus`: the beam-induced background of one bunch crossing from each beam; one entry per BIB primary particle, with all the hits it produced;
- `bib_ipp` (optional): the incoherent-pair-production background of the same crossing;
- `bib_combined`: the three merged — *made automatically* by `./run_all` when missing or out of date;
- `signal`: single-muon events (a muon gun), one muon per entry.

Branches. Per entry: `nRun`, `nEvt`; the primary particle `part_pdg`, `part_q`, momentum `part_px`, `part_py`, `part_pz`, `part_e` (GeV), production vertex `part_vx`, `part_vy`, `part_vz` (mm) and time `part_t0` (ns); the number of hits `n_hit`. Per hit, as parallel vectors:

`hit_x`, `hit_y`, `hit_z`	global position (mm)
`hit_t`	time (ns)
`hit_u`, `hit_v`; `hit_du`, `hit_dv`	position in the sensor plane and its resolution (mm)
`hit_ux`,`uy`,`uz`, `hit_vx`,`vy`,`vz`	the sensor's local u and v axes (unit vectors)
`hit_px`, `hit_py`, `hit_pz`	momentum of the particle at the hit (GeV): its direction gives the hit's direction
`hit_mcp`	PDG code of the particle the simulation credits the hit to
`hit_id0`	sensor identifier (below); `hit_index`: hit number

`hit_id0` packs `system:5, side:-2, layer:6, module:11, sensor:8` (bits, from the lowest): system = `id0 & 0x1F` (1 VXD barrel, 2 VXD endcap, 3 IT barrel, 4 IT endcap, 5 OT barrel, 6 OT endcap), side = `(id0 >> 5) & 0x3` (0 barrel, 1 and 3 the $+z$ and $-z$ endcaps), layer = `(id0 >> 7) & 0x3F`. More in `README.md`.

`[geometry]`: XML files describing the geometry of the detector (`main`, `vertex`, `inner_tracker`, `outer_tracker`); the package reads from them the position and the sensitive area of every layer.